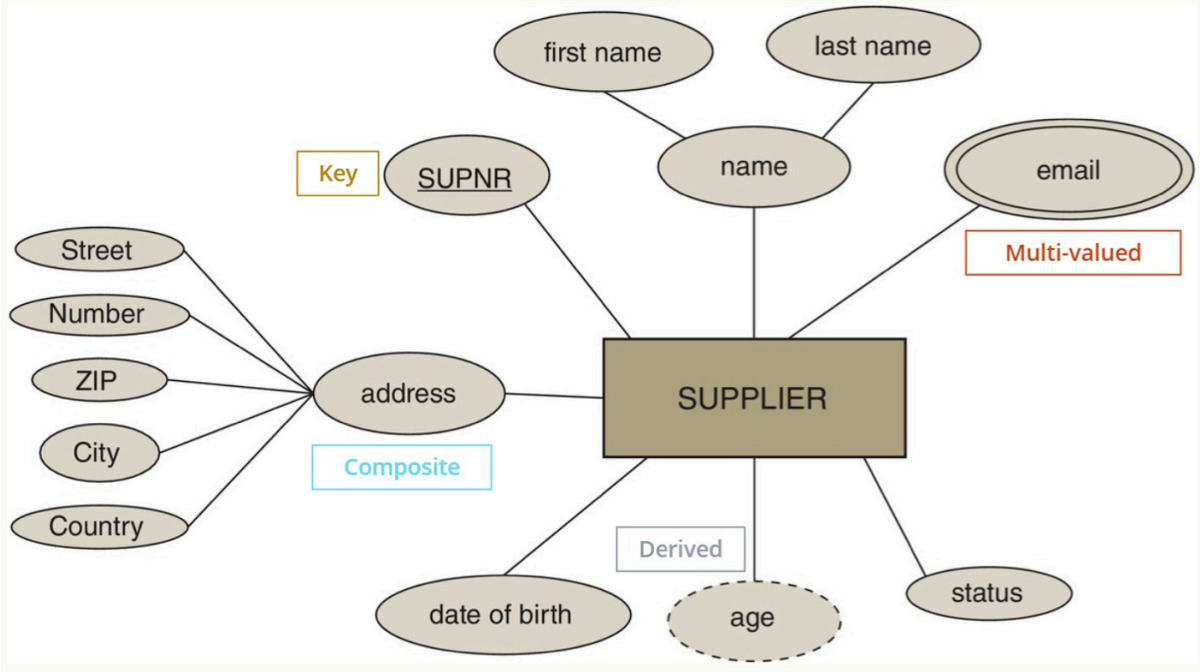
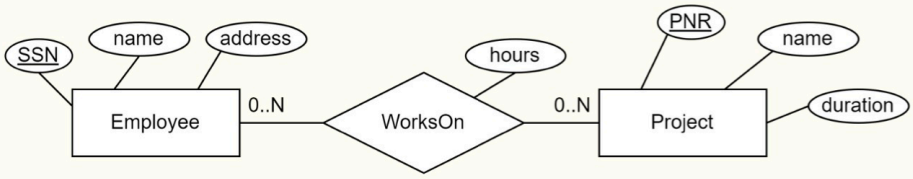
ER-diagrams

▼ Attribute types

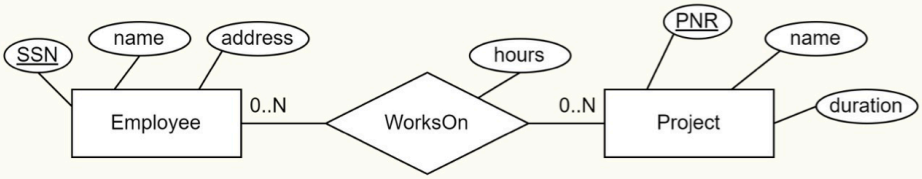


- **<u>Underlined</u>** = PRIMARY KEY
- ER Models cannot show candidate keys
    - They must be noted somewhere else! They must still be UNIQUE in the SQL DDL

▼ Relationship types




```sql
CREATE TABLE WorksOn (
   EmployeeID INT,       -- role (key of Employee)
   ProjectID INT,        -- role (key of Project)
   hours INT NOT NULL,   -- attribute
   PRIMARY KEY (EmployeeID, ProjectID),
   FOREIGN KEY (EmployeeID) REFERENCES Employee (SSN),
   FOREIGN KEY (ProjectID) REFERENCES Project (PNR)
);
```

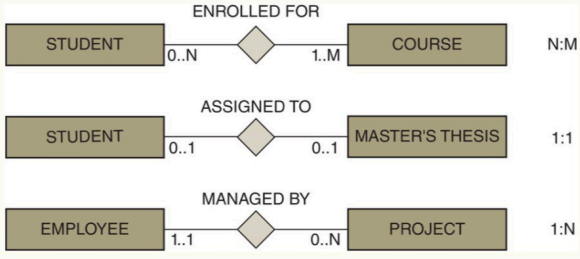


Alternative way to reference foreign keys:

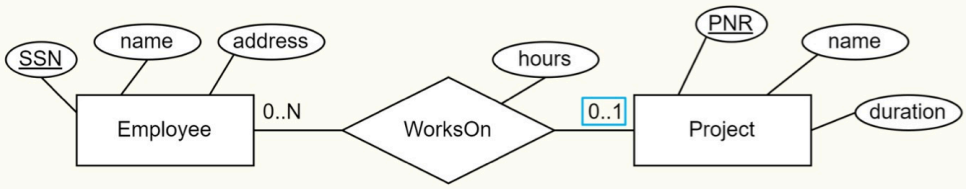
```sql
CREATE TABLE WorksOn (
   EmployeeID INT REFERENCES Employee (SSN),
   ProjectID INT REFERENCES Project (PNR),
   hours INT NOT NULL,
   PRIMARY KEY (EmployeeID, ProjectID)
);
```

▼ Cardinalities

- Relationships always have cardinalities
  - Minimum: 0 or 1
  - Maximum: 1 or N / M / L / * / ...
- Read:
  - Entity (ignore) Relationship Cardinality Entity
  - Student has to enroll in at least one course

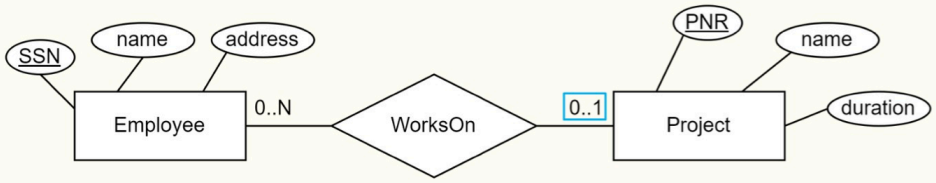


- Cardinalities impact the resulting table structure



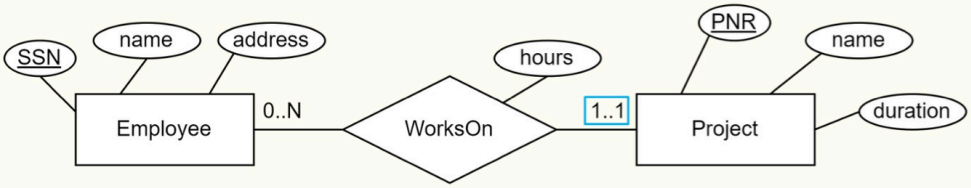
- An Employee can work on at most one Project for a set number of hours

```
CREATE TABLE WorksOn (
   EmployeeID INT REFERENCES Employee (SSN),
   ProjectID INT NOT NULL REFERENCES Project (PNR),
   hours INT NOT NULL,
   PRIMARY KEY (EmployeeID) -- each Employee only appears once
);
```



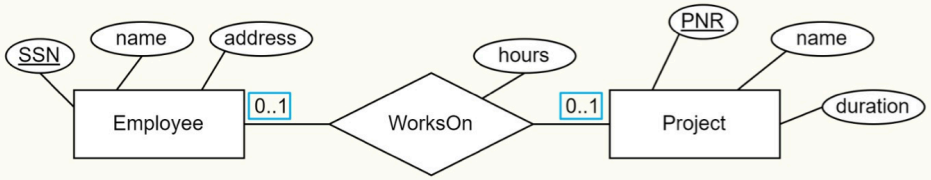
- Employee can only work on at most one Project

```
CREATE TABLE Employee (
   SSN INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   address VARCHAR(100) NOT NULL,
   PNR INT REFERENCES Project (PNR) -- WorksOn 0..1 (allows it to be NULL)
);
```

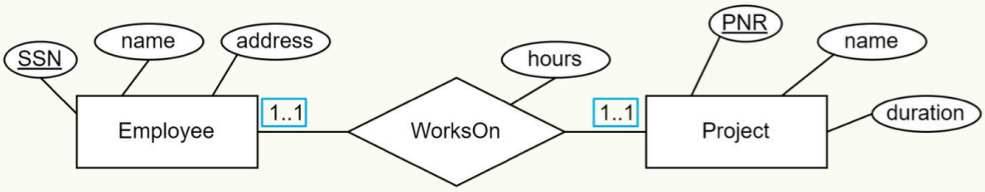


- An Employee has to work on exactly one Project

```
CREATE TABLE Employee (
   SSN INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   address VARCHAR(100) NOT NULL,
   PNR INT NOT NULL REFERENCES Project (PNR), -- WorksOn 1..1
   hours INT NOT NULL -- WorksOn 1..1
);
```




```sql
CREATE TABLE WorksOn (
  EmployeeID INT REFERENCES Employee (SSN),
  ProjectID INT NOT NULL REFERENCES Project (PNR),
  hours INT NOT NULL,
  PRIMARY KEY (EmployeeID), -- ensures each Employee once
  UNIQUE (ProjectID) -- ensures each Project once
);
```

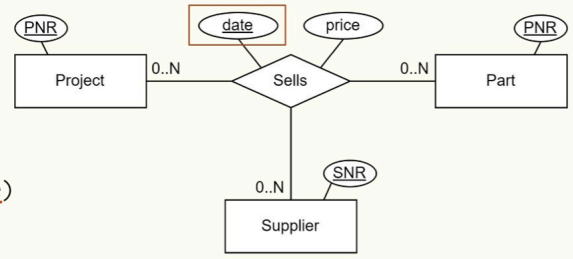



```sql
CREATE TABLE Employee (
  SSN INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  address VARCHAR(100) NOT NULL,
  PNR INT NOT NULL
  REFERENCES Project (PNR),
  hours INT NOT NULL
);
```

```sql
CREATE TABLE Project (
  PNR INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  duration INT NOT NULL,
  SSN INT NOT NULL
  REFERENCES Employee (SSN),
);
```

▼ Ternary Relationships

```sql
CREATE TABLE Sells (
  Proj_nr INT REFERENCES Project (PNR),
  Supp_nr INT REFERENCES Supplier (SNR),
  Part_nr INT REFERENCES Part (PNR),
  date DATE,
  price FLOAT NOT NULL,
  PRIMARY KEY (Proj_nr, Supp_nr, Part_nr, date)
);
```

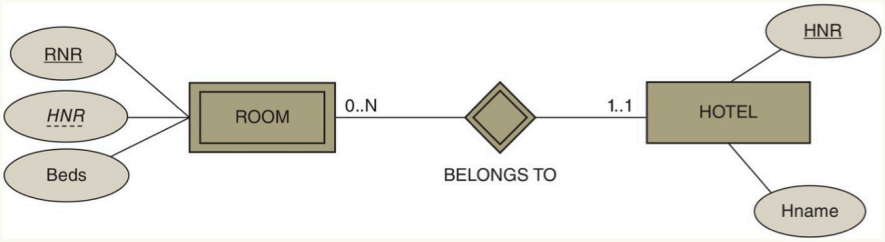


▼ Weak Entity Types

- Weak entities belong to another entity

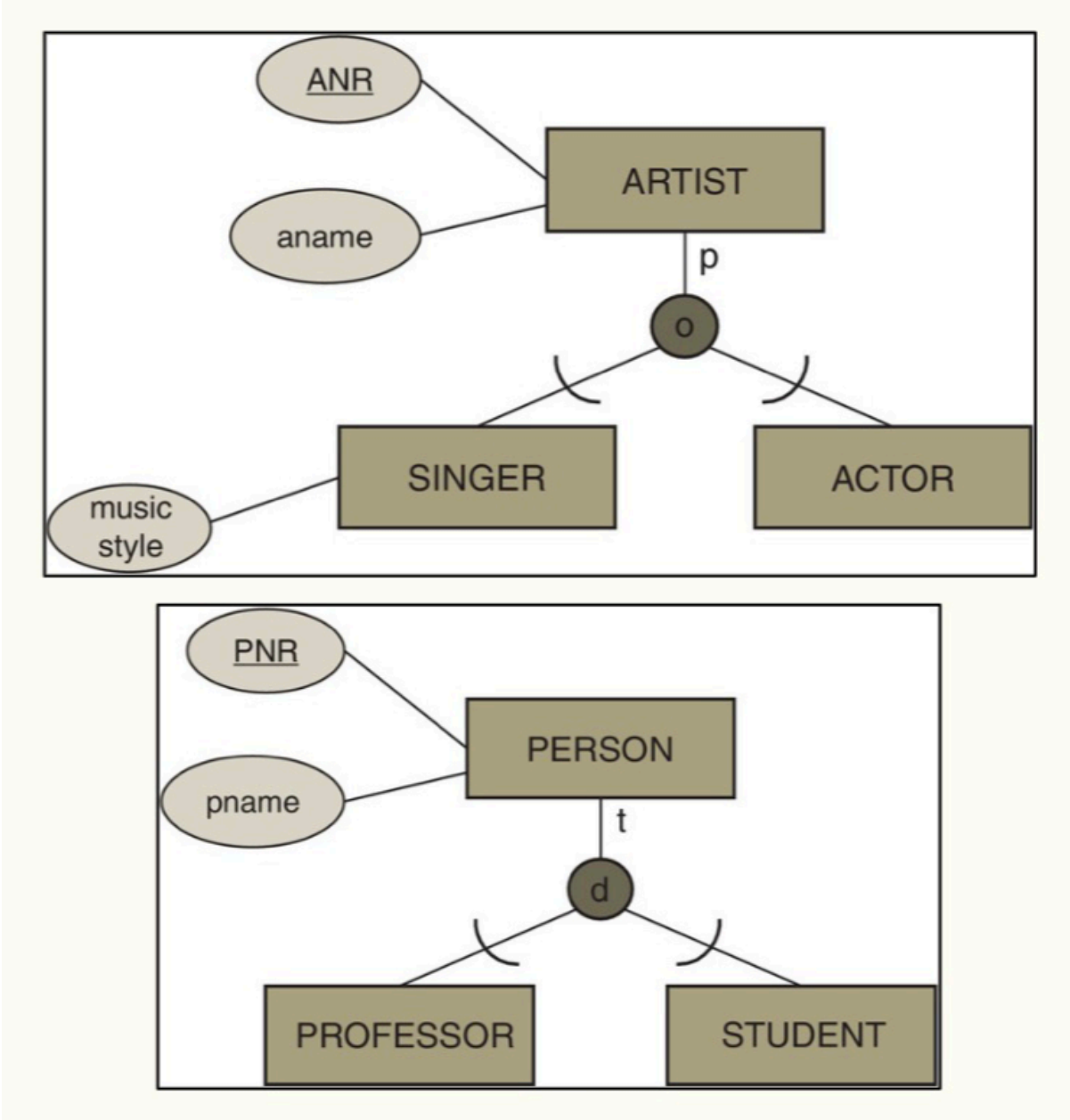
- They do not have a proper key, but include “parent” key
- They have a 1 .. 1 participation in the relationship
- If “parent” is deleted, so is the “child”

```
CREATE TABLE Room  (
   RNR INT,  -- Should not be a sequence!
   HNR INT REFERENCES Hotel (HNR),
   Beds INT NOT NULL,
   PRIMARY KEY (HNR, RNR)
);
```



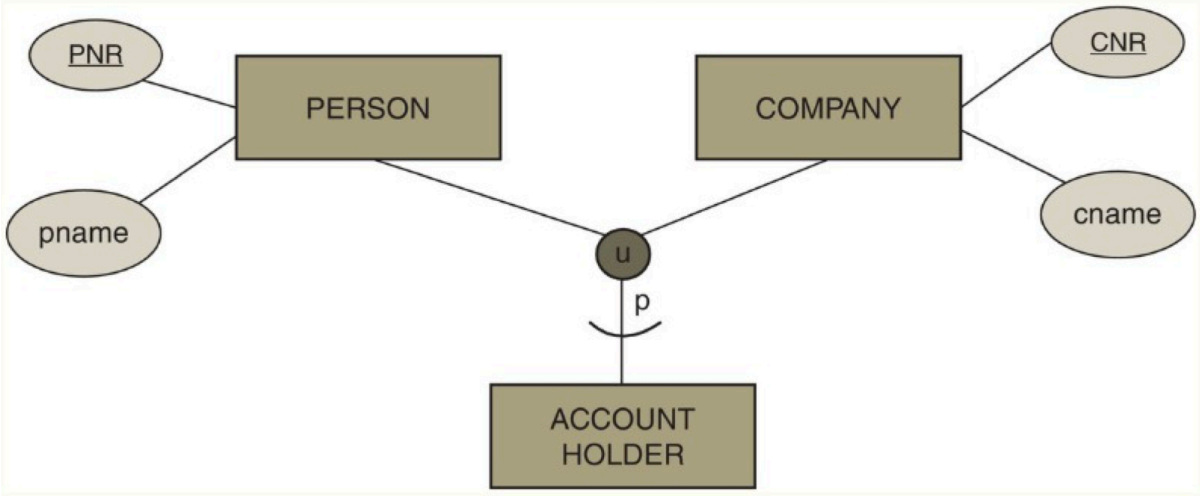
▼ Specialization / Generalization

- Partial (P): An entity does not have to be either option
- Total (T): An entity has to choose either
- Disjoint (D): An entity can not be both
- Overlapping (O): An entity can be both



▼ Categorization

- Grouping of otherwise unrelated entities




```sql
CREATE TABLE AccountHolder (
   AcctID INT PRIMARY KEY
);

CREATE TABLE Person (
   PNR INT PRIMARY KEY,
   pname VARCHAR NOT NULL,
   AcctID INT [NOT NULL]
   REFERENCES AccountHolder (AcctID)
);
```

NOT NULL = Total
NULL = Partial